

My Content

Système de recommandation d'articles

MVP

Contexte du projet

L'objectif

My Content veut encourager la lecture en recommandant des contenus pertinents.

Fonctionnalité critique du MVP :

"En tant qu'utilisateur, je reçois une sélection de 5 articles."

Les contraintes

- Pas de données utilisateurs propres
- Utilisation du dataset Globo.com (Kaggle)
- Gestion des nouveaux utilisateurs
- Gestion des nouveaux articles
- Déploiement serverless (AWS Lambda)

Les données : Globo.com

Métrique	Valeur
Interactions (clics)	2 988 181
Utilisateurs uniques	322 897
Articles cliqués	46 033
Articles en base	364 047
Catégories	461
Période	Octobre - Novembre 2017
Clics moyen par utilisateur	9.3
Taux de couverture articles	12.6%

3 fichiers sources

385 fichiers CSV (clics)
Métadonnées articles
Embeddings (250 dim.)

Matrice creuse

Densité : 0.02%
Longue traîne : le top 10% des
articles = 94% des clics

Les embeddings articles

Chaque article est représenté par un vecteur de 250 nombres, calculé par un réseau de neurones appelé CHAMELEON. Deux articles sur le même sujet ont des vecteurs proches. On ne dispose pas du texte des articles, mais ces vecteurs capturent le contenu sémantique.

Réduction de dimension (ACP)

250 dimensions réduites à 50 via ACP.
40 composantes capturent 90% de la variance.
Gain : 700 Mo réduit à 140 Mo pour le déploiement.

Couverture

364 047 articles ont un embedding.
100% des articles cliqués sont couverts.
Permet le content-based même sur les articles jamais cliqués.

4 approches de modélisation testées

1. Popularité

Baseline

2. Content-Based

Similarité cosinus

3. Collaboratif (ALS)

Factorisation matricielle

4. Hybride

CB + ALS combinés

Modèle 1 : Popularité (baseline)

Principe

On recommande les 5 articles les plus cliqués par tout le monde, en excluant ceux que l'utilisateur a déjà lus.

C'est la baseline : si nos modèles ne font pas mieux que ça, ils ne servent à rien.

Avantages

- Aucun historique nécessaire
- Instantané, pas de calcul
- Fonctionne pour tout le monde

Limites

- Zéro personnalisation
- Toujours les mêmes 28 articles
- Pas de découverte

Modèle 2 : Content-Based (cosinus)

Principe en 3 étapes

1. On prend les embeddings des articles lus par l'utilisateur et on en fait la moyenne → c'est son profil
2. On calcule la similarité cosinus entre ce profil et chaque article de la base
3. On trie par score décroissant, on exclut les déjà lus, et on prend le top 5

Avantages

- Recommande des articles jamais cliqués
- Couverture catalogue : 10 194 articles
- Gère le cold start article
- Cohérence catégorielle : 69%

Limites

- Hit Ratio faible : 0.26%
- Effet bulle de filtre
- Ne capte pas les tendances collectives
- Ne gère pas le cold start utilisateur

Modèle 3 : Collaboratif (ALS)

Principe

On décompose la matrice utilisateur-article en deux matrices de facteurs latents (utilisateurs et articles). Pour prédire si un utilisateur aimerait un article, on multiplie leurs facteurs. L'algorithme ALS (Alternating Least Squares) optimise ces facteurs par itérations successives.

Avantages

- **Meilleur Hit Ratio : 26.3%**
- Capte les patterns collectifs
- Inférence rapide (<1ms)
- Adapté au feedback implicite (clics)

Limites

- Couverture faible : 206 articles
- Ne gère pas le cold start (user ou article)
- Nécessite un historique d'interactions
- Cohérence catégorielle : 38%

Modèle 4 : Hybride

$$\text{score} = \alpha \times \text{score_ALS} + (1 - \alpha) \times \text{score_content_based}$$

Les scores de chaque modèle sont normalisés entre 0 et 1 (min-max), puis combinés. Le paramètre α a été optimisé par recherche en grille.

Pourquoi combiner ?

- ALS : précis mais peu diversifié
- Content-Based : diversifié mais peu précis
- **Hybride : le meilleur des deux**
- Cold start couvert par le CB

Résultat de la recherche alpha

Alpha optimal = 1.0 (voir 0,7 – 0,8)

Le ALS domine tellement en prédiction que le hit ratio est maximisé sans le CB.

Mais l'architecture hybride reste pertinente : elle permet le fallback vers le CB ou le cold start quand l'ALS ne peut pas répondre. (et de jouer avec l'alpha pour ajouter de la diversité)

Gestion du cold start

Nouvel utilisateur

1. Aucun historique → fallback cold start

$Score = 0.4 \times \text{récence} + 0.6 \times \text{popularité}$

2. Dès le 1er clic → content-based

Articles similaires à ce qu'il vient de lire

3. Après 5-10 clics → hybride complet

Le collaboratif a assez de données pour prédire

Nouvel article

1. Calculer son embedding

Via le module ACR de CHAMELEON ou un modèle NLP équivalent

2. Le content-based le recommande immédiatement

Similarité cosinus avec les profils existants

3. Le collaboratif le prend en compte après les premiers clics

Il entre dans la matrice d'interaction

Métriques d'évaluation

Protocole : split temporel. Le dernier clic de chaque utilisateur sert de test. Évaluation sur 5 000 utilisateurs.

Hit Ratio

Le vrai article lu est-il dans le top 5 recommandé ? C'est le taux de bonne réponse.

Mean Reciprocal Rank

Quand le bon article est trouvé, à quel rang est-il ?
Plus c'est haut, mieux c'est.

Couverture

Combien d'articles distincts sont recommandés au total ?
Mesure la diversité du modèle.

Résultats comparatifs

Modèle	Hit Ratio	MRR	Couverture	Cohérence cat.
Popularité pure	1.70%	0.61%	28	42%
Cold Start (récence + pop)	1.70%	0.61%	28	42%
Content-Based (cosinus)	0.26%	0.18%	10 194	69%
Collaboratif (ALS)	26.28%	20.86%	206	38%
Hybride (alpha = 1.0)	26.28%	20.86%	206	38%

Le collaboratif domine en prédiction (26% de hit ratio). Le content-based excelle en diversité (10 194 articles, 69% de cohérence). Les deux approches sont complémentaires : l'une prédit, l'autre diversifie.

Le système de recommandation

Fonctionnalité MVP : 5 articles recommandés par utilisateur

1

L'utilisateur ouvre l'app

Streamlit affiche son profil et historique

2

Appel au système de reco

Requête POST vers AWS Lambda

3

Modèle hybride

ALS + Content-Based + Cold Start fallback

4

5 articles retournés

Avec score de pertinence et catégorie

Fonctionnalités

- Sélecteur d'utilisateur (322 897 IDs)
- Bouton de recommandation
- Tableau des 5 articles recommandés
- Score, catégorie, nombre de mots
- Historique des articles lus
- Top catégories de l'utilisateur
- Simulation nouvel utilisateur (cold start)
- Mode local ou API Lambda

Application Streamlit

Paramètres

☐ Simuler un nouvel utilisateur (cold start)

Identifiant utilisateur

0

Articles lus : 8

Top catégories lues :

Catégorie 186 : 2 articles

Catégorie 281 : 2 articles

Catégorie 136 : 1 articles

Catégorie 209 : 1 articles

Catégorie 375 : 1 articles

Nombre d'articles

5

My Content - Systeme de recommandation d'articles

Mode API Lambda : <https://uf6ej86v531.execute-api.eu-north-1.amazonaws.com/prod/recommend>

Recommandations pour l'utilisateur 0

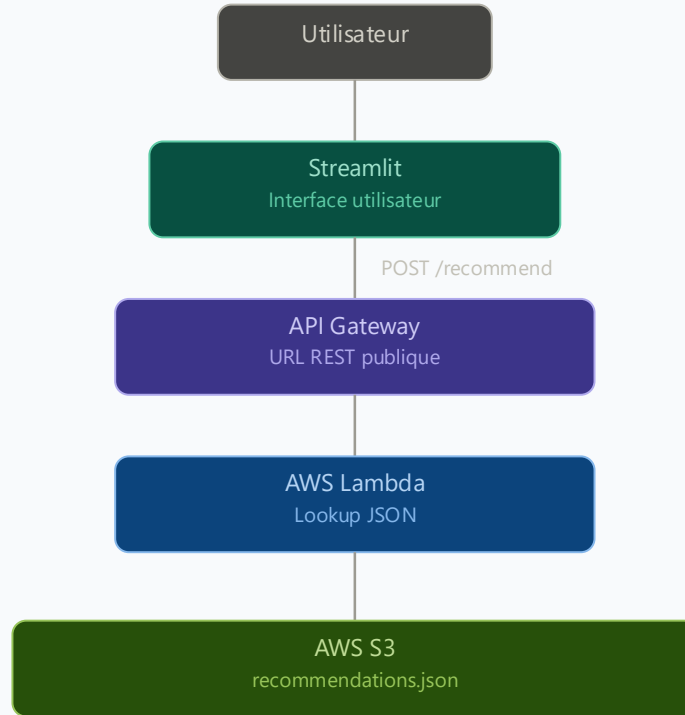
Lancer la recommandation

Rang	Article ID	Score	Catégorie	Mots	Source
1	293114	1	421	213	hybride
2	284985	0.8028	412	184	hybride
3	272218	0.461	399	238	hybride
4	160940	0.3487	281	275	hybride
5	124749	0.3432	250	192	hybride

Derniers articles lus par cet utilisateur

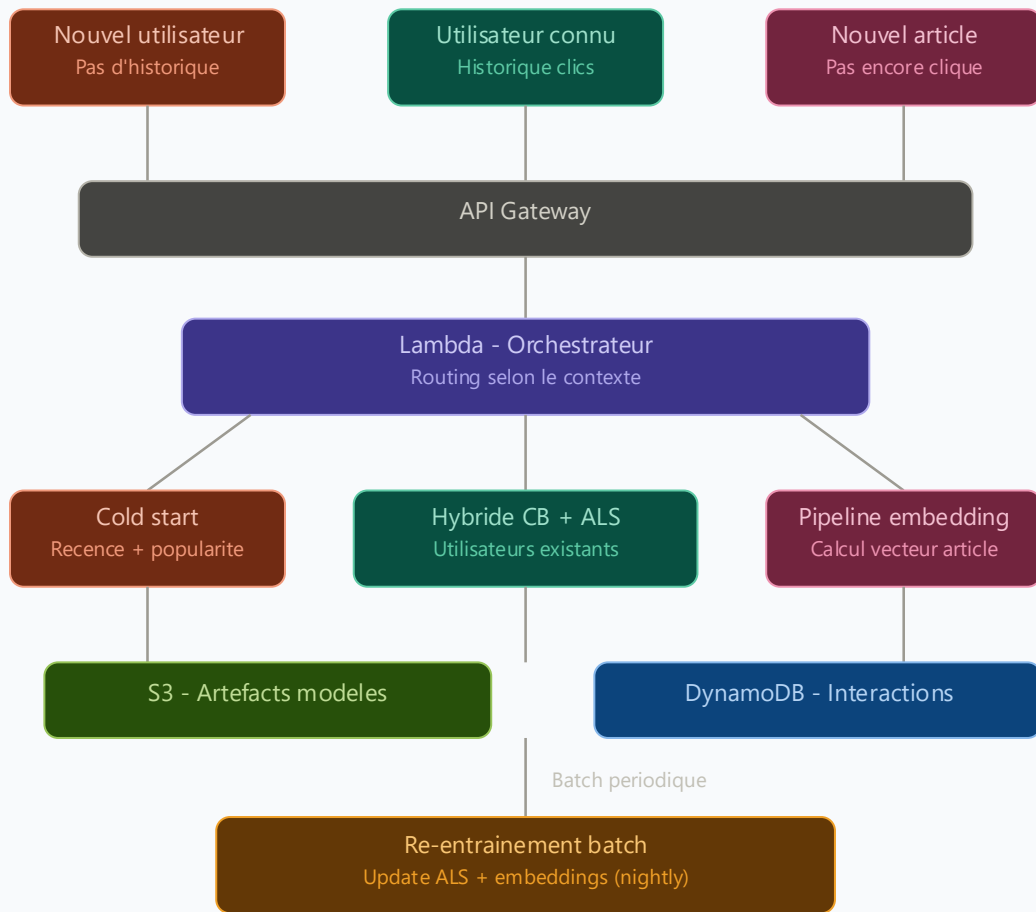
Article ID	Catégorie	Mots
87205	186	137
87224	186	100
233470	375	292
160158	281	145
313996	431	265
96755	209	211
68866	136	226

Architecture MVP (déployée)



Les recommandations sont pré-calculées (precompute.py), stockées en JSON sur S3, et servies par Lambda via API Gateway.

Architecture cible



Nouveaux utilisateurs → cold start puis transition vers hybride. Nouveaux articles → pipeline embeddings puis content-based immédiat.

Perspective : A/B Testing

L'évaluation offline (hit ratio) ne suffit pas. Il faut valider en conditions réelles que les recommandations génèrent de l'engagement.

Groupe A (50%)

Reçoit les recommandations du modèle hybride (ALS + CB).

On mesure : taux de clic, temps de lecture, nombre d'articles lus par session.

Groupe B (50%)

Reçoit les recommandations par popularité pure.

Même métriques mesurées. Au bout de quelques semaines, on garde le modèle gagnant.

Démonstration

Application Streamlit + AWS Lambda

Merci
